# Practical Svelte

Create Performant Applications with the Svelte Component Framework

—

Alex Libby

Apress®

# Practical Svelte

## Create Performant Applications with the Svelte Component Framework

**Alex Libby**

Apress®

***Practical Svelte: Create Performant Applications with the Svelte Component Framework***

Alex Libby
BELPER, UK

ISBN-13 (pbk): 978-1-4842-7373-9 ISBN-13 (electronic): 978-1-4842-7374-6
https://doi.org/10.1007/978-1-4842-7374-6

Copyright © 2022 by Alex Libby

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Managing Director, Apress Media LLC: Welmoed Spahr
Acquisitions Editor: Louise Corrigan
Development Editor: James Markham
Coordinating Editor: Nancy Chen

Cover designed by eStudioCalamar

Cover image by Clark van der Beken on Unsplash (www.unsplash.com)

Distributed to the book trade worldwide by Apress Media, LLC, 1 New York Plaza, New York, NY 10004, U.S.A. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springeronline.com. Apress Media, LLC is a California LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a **Delaware** corporation.

For information on translations, please e-mail booktranslations@springernature.com; for reprint, paperback, or audio rights, please e-mail bookpermissions@springernature.com.

Apress titles may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Print and eBook Bulk Sales web page at http://www.apress.com/bulk-sales.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub via the book's product page, located at www.apress.com/9781484273739. For more detailed information, please visit http://www.apress.com/source-code.

Printed on acid-free paper

*This is dedicated to my family, with thanks for their love and support while writing this book.*

# Table of Contents

**About the Author** ..... xiii

**About the Technical Reviewer** ..... xv

**Acknowledgments** ..... xvii

**Introduction** ..... xix

**Chapter 1: Getting Started** ..... 1

Introducing Svelte ..... 1

Setting Up a Development Environment ..... 2

So How Do We Get Svelte? ..... 3

Creating Your First Application ..... 5

Understanding What Happened ..... 8

What Makes Svelte Different? ..... 9

Browser Support ..... 13

Setting Up Our Project ..... 13

Reviewing the Changes Made ..... 15

Exploring the Structure of Our Site ..... 15

Setting the Background ..... 18

Summary ..... 19

**Chapter 2: Creating Components** ..... 21

Understanding the Makeup of Components ..... 21

Creating an Example Component ..... 22

Building the Example Component ..... 23

Reviewing the Code ..... 26

Setting Expectations ..... 27

Working Out the Architecture ..... 28

Making a Start with App.svelte ..... 31
Reviewing the Code Changes ..... 33
Setting Up the Component Index ..... 34
Creating the Header ..... 35
Setting Up the Footer ..... 37
Creating the Product Gallery ..... 39
Building the Product Card ..... 41
Reviewing the Code ..... 43
Creating a Button Component ..... 44
Adding the Cart ..... 45
Breaking Apart the Code Used ..... 48
Creating the Remaining Pages ..... 49
File Formats and Svelte – A Note ..... 49
Testing the Initial Site ..... 49
Summary ..... 50

**Chapter 3: Managing State and Data ..... 53**

Why Is Data So Important in Svelte? ..... 53
Passing and Setting Values in Code ..... 55
Exploring the World of Props ..... 56
Reviewing the Code ..... 58
Making Use of Context in Svelte ..... 59
Breaking Apart the Code ..... 61
Creating and Maintaining Svelte Stores ..... 63
Understanding the Code in Detail ..... 65
Understanding the Differences ..... 66
Adding Functionality to Our Site ..... 68
Adding Props Support ..... 69
Exploring the Changes ..... 71
Adding New Functionality Using Stores ..... 71
Breaking Apart the Code ..... 73

Exploring Alternative Data Sources ..... 74
Summary ..... 76

**Chapter 4: Controlling Logic ..... 77**

Creating If-Else Conditions ..... 77
Breaking Apart the Code ..... 80
Implementing Each Conditions ..... 81
Exploring the Code ..... 83
Removing the Right Item ..... 84
Breaking Apart the Code in Detail ..... 87
Fetching Content ..... 88
Understanding the Changes Made ..... 92
Exploring Existing Code in the Demo ..... 93
Updating the Shop ..... 94
Adding Login Functionality to Our Site ..... 94
Loading Products ..... 97
An Epilogue – Improving the Placeholder Message ..... 100
Summary ..... 102

**Chapter 5: Managing Events ..... 103**

The Theory of Reactivity ..... 103
Choosing from Different Event Types ..... 105
Creating Events ..... 106
Understanding the Changes ..... 108
Modifying Event Handlers ..... 109
Exploring the Changes in Detail ..... 113
Creating Events in Components ..... 114
Forwarding Events ..... 114
Breaking Apart the Changes ..... 116
A Point of Note ..... 117

Life Cycling Events in Svelte ..... 118
Working with onMount ..... 119
Exploring beforeUpdate ..... 122
Understanding How It Works ..... 123
Implementing Actions ..... 124
Reworking the Code ..... 126
Understanding What Happened ..... 127
Adding Functionality to Our Site ..... 128
Adding a Modal ..... 129
Exploring Changes Made to Code ..... 131
Exploring the Examples ..... 131
Using the Plugin – An Epilogue ..... 132
Summary ..... 133

**Chapter 6: Binding Elements ..... 135**

Creating Forms in Svelte ..... 135
Catering for Different Form Elements ..... 136
Working with Numeric Inputs ..... 136
Using Checkbox Inputs ..... 138
Handling Multiline Text Area Inputs ..... 138
Selecting Choices ..... 139
Dealing with Content Editable Fields ..... 139
Binding and Managing Other Events ..... 140
Building a Composite Example ..... 141
Breaking the Code Apart ..... 145
Managing Context ..... 146
Understanding the Changes ..... 150
Adding a Contact Form to Our Demo ..... 150
Introducing Svelte-Yup ..... 151
Laying the Groundwork ..... 152
Building the Contact Form ..... 153
Summary ..... 158

**Chapter 7: Adding Styles ..... 159**
Making Some Small Adjustments ..... 159
Understanding Styling in Svelte ..... 161
Adding Styling to the Front-End Demo ..... 161
Updating the Global Stylesheet ..... 162
Styling the Pages ..... 164
Styling the Header and Footer ..... 167
Styling the Buttons ..... 168
Styling the Product Pages and Cart ..... 169
Using a Style Preprocessor ..... 173
Breaking the Code Apart ..... 176
Taking Things Further ..... 177
Summary ..... 179

**Chapter 8: Unit Testing and Svelte ..... 181**
Deciding Our Strategy ..... 181
Browser Support ..... 183
Setting Up the Test Environment ..... 183
Understanding the Changes Made ..... 186
Creating an Example Test ..... 187
A Note About File Structure ..... 190
Applying Principles to Our Project ..... 190
Writing Tests for the Home Page ..... 191
Testing the Remaining Content Pages ..... 195
Testing the Product Pages and Cart ..... 199
Debugging Svelte ..... 204
Using Lighthouse ..... 207
Summary ..... 209

**Chapter 9: Deploying Svelte ..... 211**
Doing Final Checks ..... 211
Understanding the Deployment Process ..... 214

Setting Up the Hosting Provider ..... 215
Checking the Default Branch ..... 218
Deploying Content to Production ..... 219
Breaking Apart the Changes ..... 222
Adding a Custom Domain Name ..... 222
Breaking Apart the Code Changes ..... 226
Taking It Further ..... 227
Summary ..... 231

**Chapter 10: Working with External Libraries ..... 233**

Importing Libraries into Svelte ..... 233
Testing Compatibility with Svelte ..... 235
Exploring the Code in Detail ..... 240
Working Client Side vs. Server Side ..... 241
Why Is Svelte Different? ..... 242
Adding a Payment Processor ..... 242
Getting Prepared ..... 243
Taking a First Look at the PayPal Demo ..... 245
Adding PayPal to Our Front-End Site ..... 247
Testing the Result ..... 251
Reviewing the Changes Made ..... 254
What About React? ..... 255
Breaking Apart the Changes ..... 258
Summary ..... 258

**Chapter 11: Animating Svelte ..... 261**

Auditing the Site ..... 261
Learning a Language ..... 262
Understanding How the Code Works ..... 267
Reviewing the Types of Animation Options ..... 268

Applying Easing Effects ..... 269
Creating Custom Easing Effects in JavaScript ..... 269
Using Styles to Create Custom Easing Effects ..... 272
Taking It Further ..... 274
Using Svelte Libraries ..... 275
Applying Animation to Our Project ..... 276
Adding Animation to the Banner ..... 277
Updating the Cart Action ..... 278
Iterating Through Pictures ..... 282
Summary ..... 285

**Chapter 12: Adapting for Sapper ..... 287**
Introducing... Sapper? ..... 287
Migrating Our Project – Setting Expectations ..... 288
Setting Up the Initial Site ..... 289
Exploring the Code in Detail ..... 291
Understanding the Structure of a SvelteKit Site ..... 292
Updating the Layout ..... 293
Adding Pages ..... 296
Adding Product Gallery ..... 297
Adding Product Page ..... 300
Migrating Styles ..... 303
Breaking Apart the Code ..... 306
Testing the Site and Next Steps ..... 307
Summary ..... 309

**Appendix: Getting Help ..... 311**

**Index ..... 313**

# About the Author

**Alex Libby** is a front-end engineer and seasoned computer book author who hails from England. His passion for all things open source dates back to the days of his degree studies, where he first came across web development and has been hooked ever since. His daily work involves extensive use of React, Node.js, JavaScript, HTML, and CSS. Alex enjoys tinkering with different open source libraries to see how they work. He has spent a stint maintaining the jQuery Tools library and enjoys writing about open source technologies, principally for front-end UI development.

# About the Technical Reviewer

**Rami Morrar** is a self-taught programmer and has coding experience in languages such as C# and C++ for over three years. He has made different coding projects, such as a gaming tutorial website, several desktop applications, and even games in the Unity game engine. He is currently working on his own independent game project in MonoGame, set to be released next year. He has an abundance of charisma for programming and game development, and talks about both on his blog, and tutorials he has created for the MonoGame framework. He is also cowriting a sequel to the book *MonoGame Mastery*. In his free time, he likes to play games and look at cool new projects by other indie developers.

# Acknowledgments

Writing a book can be a long but rewarding process; it is not possible to complete it without the help of other people. I would like to offer a huge vote of thanks to my editors – in particular, Nancy Chen and Louise Corrigan; my thanks also to Rami Morrar as my technical reviewer, James Markham for his help during the process, and others at Apress for getting this book into print. All have made writing this book a painless and enjoyable process, even with the edits!

My thanks also to my family for being understanding and supporting me while writing. I frequently spend lots of late nights writing alone, or pass up times when I should be with them, so their words of encouragement and support have been a real help in getting past those bumps in the road and producing the finished book that you now hold in your hands.

Lastly, it is particularly poignant that the book was written at a time when the world has faced global events of an unprecedented nature; it was too easy to think about those who lost the greatest thing we as humans could ever have. Having a project to work on – no matter how simple or complex it might be – helped me get through those tough times, and with the hope that we face a new, improved, and hopefully better future.

# Introduction

*Practical Svelte* is for people who want to quickly create ecommerce sites that are efficient and fast, using the upcoming Svelte framework and associated tools.

This project-oriented book simplifies the setting up of a Svelte site as a starting point before beginning to explore the benefits of using Svelte in an ecommerce environment and developing it into an ecommerce offer that we can customize according to your needs. It will equip you with a starting toolset that you can use to create future projects, incorporate into your existing workflow, and that will allow you to take your websites to the next level.

Throughout this book, I'll take you on a journey through constructing the front-end UI for our example site. We will touch on subjects such as adding data sources, creating the catalog, and implementing a payment function and more – showing you how easy it is to develop simple ecommerce sites that we can augment later quickly. With the minimum of fuss and plenty of practical exercises, we'll focus on topics such as managing data and state, styling, creating components, and more – right through to producing the final result viewable from any browser!

Svelte uses nothing more than standard JavaScript, CSS, and HTML, three of the most powerful tools available for developers: you can enhance, extend, and configure your site as requirements dictate. With Svelte, the art of possible is only limited by the extent of your imagination and the power of JavaScript, HTML, and Node.js.

*Practical Svelte* gets you quickly acquainted with creating and manipulating ecommerce sites using tools familiar to all developers. It's perfect for website developers who are already familiar with JavaScript and are keen to learn how to leverage the Svelte framework. You may also be a developer for whom time is of the essence and simplicity is key; you need to produce efficient and properly optimized content in modern browsers using tools already in your possession.

CHAPTER 1

# Getting Started

*Wait, this new framework has a runtime? Ugh. Thanks, I'll pass.*

Let me begin with a single fact: we're shipping too much code to our users.

Yes, it might well be the case that tools, such as React and Angular, are all the rage nowadays. But most of them have one thing in common – they each have a runtime library that must run when rendering websites using these tools. Sure, this might be a necessary evil when creating component-driven websites, but it's okay, as everyone does it, right?

It's not okay. We may think it's acceptable to run a 100Kb+ runtime with the likes of React, but it's not just the code we're using. We have to consider the bandwidth, server resources, external resources (such as third-party libraries or assets), and so on – it starts to add up!

Can we do anything about this? We can, by answering this question: *What if our framework didn't run in the browser?*

## Introducing Svelte

Asking this question might seem like a real shocker when we've been used to the likes of React, but it is possible – let me introduce you to Svelte!

Created by Rich Harris in 2016, it was designed to prove that you don't need to create lots of extra component code but could write code that is much closer to vanilla JavaScript. Following this mantra means you can reduce the amount of code you need overall; in a sense, Svelte follows the principle of not reinventing the wheel when you already have perfectly good markup that can do the job. However, what makes it work is that it compiles code into reusable JavaScript modules – **without the need to operate the framework at runtime**.



© Alex Libby 2022
A. Libby, *Practical Svelte*, https://doi.org/10.1007/978-1-4842-7374-6_1

Not running a framework at runtime is significant: we don't pay the cost of shipping a sizable runtime library (and yes, tools like React get bigger, not smaller), plus our application will be super fast, without the abstraction you otherwise get with other tools.

Throughout this book, I will take you on a journey through creating a simple front-end ecommerce store that we can use on any small to medium online retail site. We'll start with a quick review of how Svelte works in this chapter but then swiftly move through creating the various components required for our store, as well as sourcing data. We'll take a look at a host of different topics on the way, such as managing events, styling, and the like – with the focus on creating our store but learning about the various parts that make up Svelte.

We have a lot to cover – so without further ado, let's dive in. As with all things programming, we will be setting up our development environment, ready for working on our Svelte project.

# Setting Up a Development Environment

Okay, with the introductions over, it's time to get to business!

Throughout this book, we will be building up a sample ecommerce storefront, which we can later attach to a back-end payment system such as Shopify. We, of course, need Svelte (after all, that is what this book is about!), but we also need to avail ourselves of a few tools, so without further ado, here is a list of what we need:

- A copy of Svelte – there are two ways to get it; we will explore this shortly.
- A text editor, such as Atom or Visual Studio Code – I would recommend taking a look at `https://svelte.dev/blog/setting-up-your-editor`, which details ways to set up some of the more popular editors for use with Svelte.
- We also need Node.js – not only for Svelte but also for other tools such as webpack. Go ahead and download a version suitable for your platform from `www.nodejs.org` – choosing the default settings will be sufficient for this book.
- A local web server such as Apache (`www.apachefriends.org`) – for Linux users, you may find this already available in your distro. For this, default settings will suffice; it's not necessary to add SSL support.

- A hosting account from a service such as Vercel, Surge, or Now for publishing our site.
- A Git account, with either GitLab or GitHub. I usually use GitHub (as I already have lots of GitHub repositories); please feel free to adapt for GitLab if preferred.
- An optional extra is a custom domain name – this isn't obligatory, but it will give our demo a little extra polish!

---

If you happen to use the Prettier tool in your browser, then I would strongly recommend checking out `www.rockyourcode.com/prettier-and-es-lint-setup-for-svelte-js/` and in particular for details on setting up Prettier to work with Svelte. The Svelte plugin for Prettier is available at `https://github.com/sveltejs/prettier-plugin-svelte`.

We need a couple of things later in the book, but we will go through details nearer the time. We can download any other Node.js packages as and when needed; I will highlight this at the appropriate point.

---

Okay, let's move on. I've touched on the fact that we need Svelte, but asked one question: how *do* we get a hold of and install it? There are a couple of options open to us, so let's take a look at them in greater detail.

## So How Do We Get Svelte?

One of the great things about Svelte is that as a JavaScript-based framework, it is straightforward to set it up – there are several options open to us:

1. Use the REPL at `https://svelte.dev/repl/hello-world?version=3.31.2` to familiarize ourselves with Svelte – this has several examples at `https://svelte.dev/examples`, which we can tweak to suit our needs.

---

The example URL will redirect to show the first in the list; you can use that list to select an appropriate option.

---

2. When we outgrow the REPL option, we can download and install a customized version from the Svelte GitHub site, using Node.js/NPM.

3. We can also use degit, a project scaffolding tool available from https://github.com/Rich-Harris/degit, to set up a new Svelte project. Using this route would typically require entering these commands:

```
npx degit sveltejs/template my-svelte-project
cd my-svelte-project
npm install
npm run dev
```

4. If you're something of a traditionalist like me, you can also install directly using NPM too – this is perfectly fine.

---

We will cover Svelte installation in more detail later when we set up our base site for this book.

---

By default, Svelte comes with the Rollup tool to manage the bundling of ES modules; we can swap it out for different integrations, depending on what tools we already use.

For example, you might choose to work with webpack instead – Svelte has a plugin available at https://github.com/sveltejs/svelte-loader to support this system. Alternatively, you can use the `sirv-cli` tool to preview assets directly – this is available from https://github.com/lukeed/sirv.

---

All of the demos in this book were written for Windows, as this is the author's usual development platform. Please adapt if you are using macOS or Linux on your PC.

---

Now that we have our development environment set up, let's try creating Svelte code using the REPL tool.

# Creating Your First Application

We could spend ages talking through how Svelte works, but that's not so interesting – instead, let's dive in and use the REPL tool to start manipulating some simple Svelte code! It's a great way to learn, mainly as Svelte code is structured in a format that closely resembles how you might write a CodePen demo. We will revisit this structure shortly in more detail, but for now, let's dive in and take a quick look at a typical "Hello World" app to get up to speed with using Svelte.

**BREAKING INTO USING REPL**

To get acquainted with Svelte, using the REPL tool, follow these steps:

1. First, crack open your browser and navigate to `www.svelte.dev`. Click the REPL link at the top right of the page.

---

The keen-eyed among you will note that you could equally browse directly to the REPL tool once you know the link. Browsing to this URL will work, but be aware though that it includes a version-specific reference to Svelte. You may end up invertedly browsing to and using an older version of Svelte if you're not careful!

---

2. Once at the page, you will see a split code window – on the left is an open file, `App.svelte`, with the Result, JS output, and CSS output tabs on the right. The tool comes with a version of the typical "Hello World" app running, which we can see in Figure 1-1.

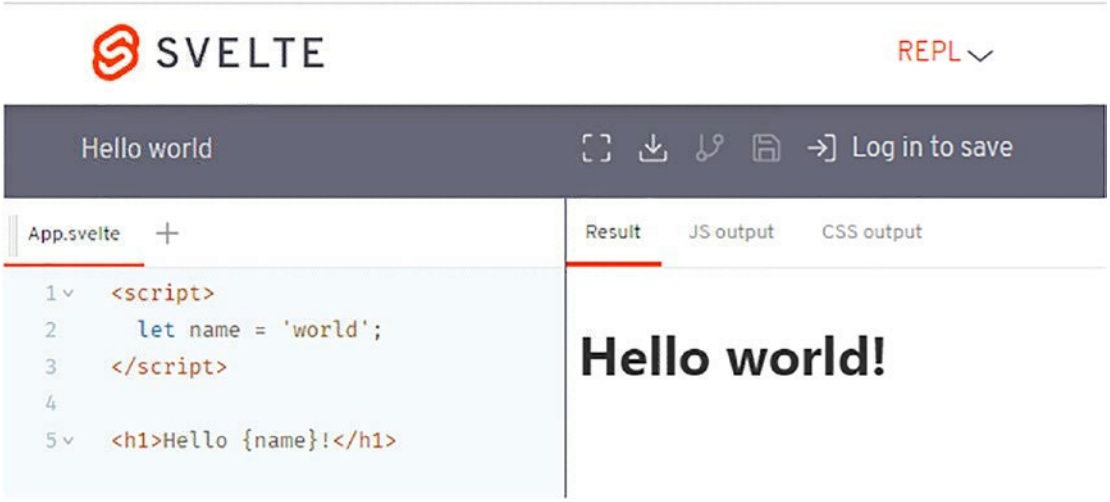


***Figure 1-1.*** *A screenshot of the REPL tool for Svelte*

3. Try changing the value assigned to the name from `'world'` to your name – see how it changes automatically on the right?

4. Let's add a little color to the result – after the opening `<script>` tag, insert the code as highlighted:

```
<script>
  let name = 'Alex';
</script>
<h1>Hello {name}!</h1>

<style>
  h1 { color: purple; }
</style>
```

5. The results shown on the right will automatically update, so we end up with "Hello…" in a purple font.

6. Okay, let's ramp things up a little. Inside the script block, leave a blank line after the let name= entry, and add this:

```
<script>
  let name = 'world';

  function handleClick() {
    alert('no more alerts');
  }
</script>
```

7. Next, add in this button code immediately after the <h1> tag:

```
<h1>Hello {name}!</h1>
<button on:click|once={handleClick}>
  Click me
</button>
```

8. Try clicking the "Click me" button – what do you get? If all is well, we should see a dialog appear (Figure 1-2, shown overleaf).

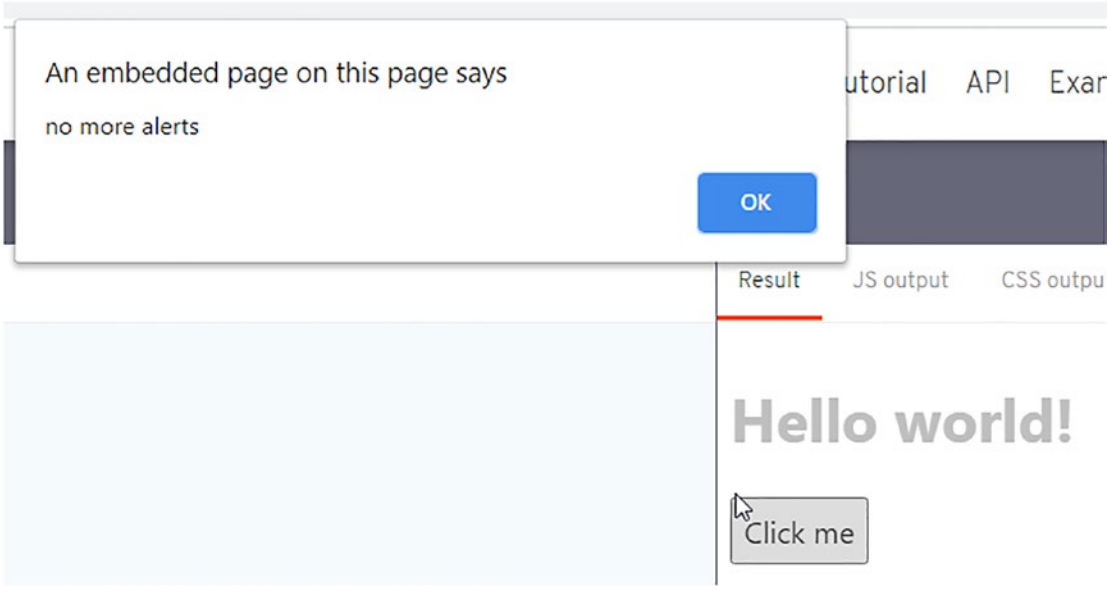


***Figure 1-2.*** *Clicking the button shows our first dialog box*

You've now completed your first Svelte demo – easy, right? Most of the code we've used is plain vanilla JavaScript, but with some key differences: let's take a moment to review what we've covered in this demo in more detail.

# Understanding What Happened

So, what did we achieve in that last demo?

It is fair to say that while it may seem to have been a trivial exercise, this was intentional – it was more about working through the constituent parts of a typical Svelte application than exploring the intricacies of the code itself! We will explore the code in more detail throughout this book, but for now, let's take a quick look at the sections that make up our application.

As a quick recap, here is the code we've just used in full:

```
<script>
  let name = 'Alex';

  function handleClick() {
    alert('no more alerts');
  }
</script>

<style>
  h1 { color: purple; }
</style>

<h1>Hello {name}!</h1>
<button on:click|once={handleClick}>
  Click me
</button>
```

You will see it's made up of three parts – we've included a `<style>` block, `<script>` block, and a markup section, which is standard for all Svelte apps.

We kicked off by browsing to the REPL tool for Svelte, which is a playground for testing code – we understood from earlier that this is perfect for getting started with Svelte, but we will later move to other tools.

The code we entered begins with a style block to add color to an H1 tag we will use in our demo. Next, we then added a script block, where we've defined a variable name to equal the text "`world`" and created a simple event handler. We then created standard markup to render that name and introduce a clickable button. There are two key differences, though – the use of parentheses to act as a placeholder for our `name` variable and the inline event handler to fire a message. If we click that message, we will get a simple alert to say "no more alerts" – it seems a little ironic to display a warning when we no longer want any alerts, but it is only a simple demo, after all!

Don't worry, though, if sections such as the event handler don't entirely make sense at this stage; it's enough to know that the format we've used is a standard way to fire events in Svelte and equates (roughly) to `<button onclick='alert("no more alerts")>Click me</button>`.

---

We will cover events in more detail later in Chapter 5 of this book.

---

Okay, let's crack on. Now that we've been introduced to Svelte, it's time for a little theory. Yes, I know: theory isn't everyone's idea of fun, but it's essential to understand how Svelte ticks and what makes it different from its competitors. Don't worry, though, I will keep it short! With that in mind, let's take a stab at answering that very question.

## What Makes Svelte Different?

If you've spent any time with frameworks over the last few years, you will no doubt have come across one feature that is common to many of the more popular frameworks in use today – virtual DOM. Say, for example, you create this React component:

```
function HelloMessage(props) {
  return (
    <div className="greeting">
      Hello {props.name}
    </div>
  );
}
```

Looks harmless, right? The object we've just created (using that code) represents how it should display on the page and the virtual DOM. If we update the state in an application with such a component, this will be re-rendered as a new object; the virtual DOM has to reconcile that against the previous version to work out what should now be displayed.

This updating all sounds good, but hold on – if we already have a *physical* DOM, why do we need a virtual one? Doesn't that add overhead to our project?

Tools such as React aim to let you build declarative, state-driven applications without worrying about state and performance – even with a virtual DOM, the speed is usually adequate for our needs. However, there are caveats to this – updating state is still a lot of work, and while features such as React's Fiber will break things into smaller chunks, it won't reduce the overall time it takes to update state.

Let's think about that for a moment – if we updated the name property to use something else, then the diffing algorithm would have to (a) check the elements, (b) enumerate through attributes, and (c) update the DOM – in this case, the text.

That might not seem much, but bear in mind that it's only point (c) that adds any value in this case – points (a) and (b) don't change!

Take, for example, this code example, taken from the Svelte blog:

```
function MoreRealisticComponent(props) {
  const [selected, setSelected] = useState(null);

  return (
    <div>
      <p>Selected {selected ? selected.name : 'nothing'}</p>
      <ul>
        {props.items.map(item =>
          <li>
            <button onClick={() => setSelected(item)}>
              {item.name}
            </button>
          </li>
        )}
      </ul>
    </div>
  );
}
```

Suppose we were to change the `props.item` value, then we would be creating a new array of virtual li elements each time – while it may not be an issue and be sufficiently fast for your needs, it's still unnecessary! We risk ending up landing ourselves with a bottleneck that becomes difficult to optimize at a later date.

What if we could achieve the same result without the use of a virtual DOM? This is where Svelte comes in – instead of compiling code at runtime (and therefore needing that runtime library), we compile at build time into pure, highly efficient imperative code.

Let's say that a typical to-do app in Svelte weighs in at about 4Kb. Compare that to production versions of React and ReactDOM (taken from a CDN), without any application code, weigh in at 128Kb. That equates to 32x the size! Granted, it may not be an entirely fair comparison, but the point is that there is still a hefty increase that we have to consider when using tools such as React. You can see the steps we have to go through when using React, compared to Svelte, in Figure 1-3.

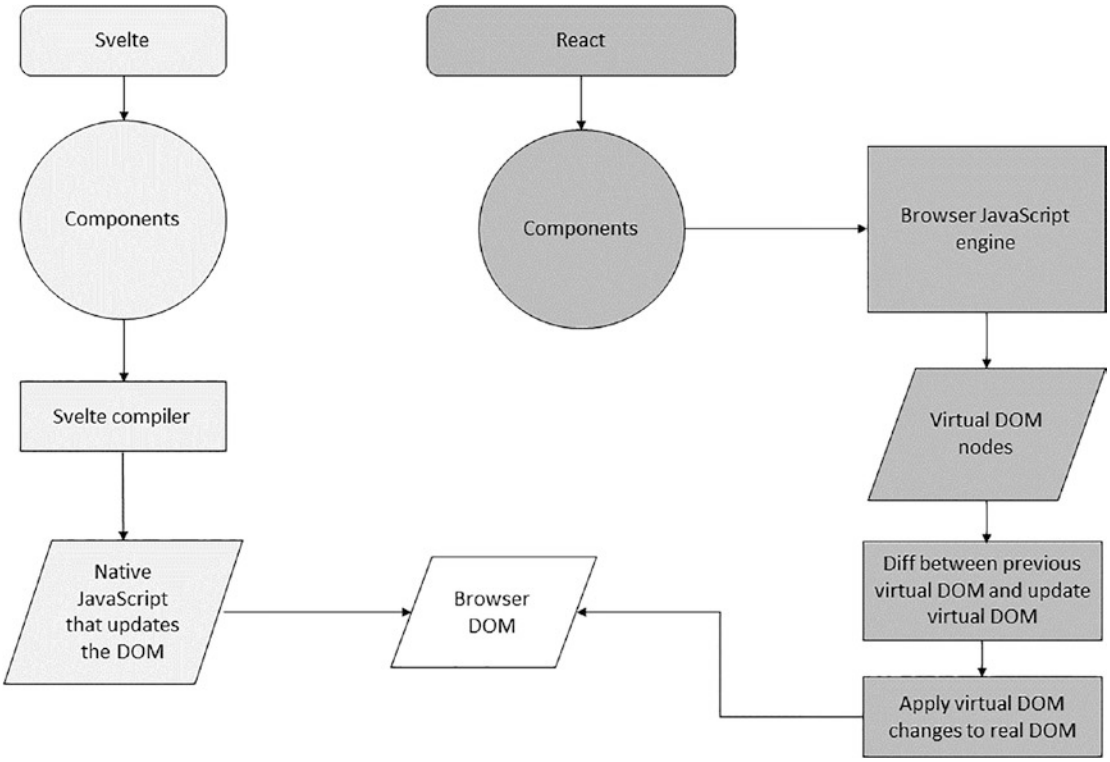


***Figure 1-3.*** *Comparing the diffing process for Svelte vs. React*

It isn't the only difference, though, when using Svelte – here are two more:

- Interoperability – if you wanted to use an NPM package, you would likely need to find one tailored to your framework, such as React, Ember, or Angular. However, if we had built that package using Svelte, we can use that package with any framework, including Svelte!
- When it comes to code splitting, it doesn't matter how many components you serve – you still have to serve a runtime such as React. It isn't an issue with Svelte; the framework is embedded in each component, which means the code is much smaller, and therefore code splitting is more effective.
- Svelte, from version 3 onward, also took a radical decision not to implement an API. It removes both the need for code that can seem unnatural and reduces the work required for garbage collection; both help keep the size of the framework much smaller than the likes of React and Angular.

Although we can build entirely usable applications using Svelte by itself, this isn't the only option – to take advantage of techniques such as server-side rendering (SSR) and code splitting (essential in today's development), we can use Sapper.

Sapper is an application framework built using Svelte, which comes preconfigured with best practices such as SSR and code splitting. Svelte developers recommend it as the preferred way to build applications. It makes developing applications simpler, and we should consider it when it comes to architecting any project using Svelte.

---

What's the deal with SvelteKit?

Sapper is undergoing a complete rewrite, as it is based on techniques that have been available for a few years and have become outdated. Its replacement is SvelteKit, which is still in active development – we'll explore this and more later in Chapter 12.

---

Okay, let's move on. We need to cover one more topic before getting stuck into code, though, which is the issue of browser support. Svelte supports most recent browsers, with at least one exception – to understand what that means for us, let's dive in and take a closer look.

## Browser Support

In an age of responsive design, I am sure there will be one question at the top of everyone's mind when using the Svelte framework. Which browsers does it support, and how will this align with my company's requirements?

Fortunately for Svelte, support is excellent – it supports most **recent**, **well-known** browsers, including mobile devices, but with one caveat: it doesn't support IE11, at least not out of the box.

IE11, I hear you say that's an old browser! Many companies still use IE11, primarily because of investment in tools and technologies such as ActiveX.

There is a solution though, of sorts – Svelte will operate in IE11, but only if we include polyfills; the trade-off is that these will bump up the overall size enormously, which means we lose the benefit of tiny, fast files! It is something to think about – can you manage transpiling for IE11, with large polyfills in use, at least until you can move away from supporting IE11?

---

For more details and some of the support issues raised around IE11, you might like to refer to GitHub issue 2621 on the Svelte repo, at `https://github.com/sveltejs/svelte/issues/2621`. Other developers also provide solutions, such as Ingo Farentholtz's IE11 solution at `https://github.com/ifahrentholz/svelte-3-ie11` or the blog post written by Mark Volkmann at `https://mvolkmann.github.io/blog/topics/#blog/svelte/supporting-ie11/`.

---

Okay, so we've been introduced to Svelte, explored what makes it different from other frameworks, and dipped our proverbial toes into the water: it's time to get stuck in! Let's dive in and take a look at what's involved in more detail.

# Setting Up Our Project

We've already covered the various ways to install Svelte. For this book, I will focus on using option 2, the custom Svelte template, available from the main Svelte site.

It has several useful files and folders already configured for us, such as `package.json` and the `src` folder, making it easier to develop our project. It's a simple process to set up the site, so let's make a start.

## DEMO – CREATING OUR INITIAL SITE

To set up our base site, ready for building into a store front end, follow these steps:

1. The first step is to install Node.js and NPM – go ahead and browse to www.nodejs.org, and then download and install a version appropriate for your platform. If prompted, please use the default settings.
2. Next, go ahead and extract a copy of the `template-master.zip` file from the code download that accompanies this book – this contains our preconfigured site, ready for installing.

---

It's also available from https://github.com/sveltejs/template/archive/master.zip.

---

3. Go ahead and extract the template-master folder from within the zip file, and save it to your PC. Then rename it to `sveltebook`. Next, fire up a Node.js terminal session, and then change the working folder to the `sveltebook` folder from step 3.

---

For the exercises in this book, I will assume the location is at the root of your C: drive. Please alter as appropriate if you would like to save it to a different location.

---

4. At the prompt, enter `npm install && npm run dev` to install and run the project. After a few moments, we should see this in the terminal session:

```
Your application is ready~! 🚀

  - Local:      http://localhost:5000
  - Network:    Add `--host` to expose

──────────────────────────── LOGS ────────────────────────────
```

5. Assuming we do, switch to your browser, and then browse to `http://localhost:5000` to see our base site in all its glory, with the text shown in Figure 1-4.

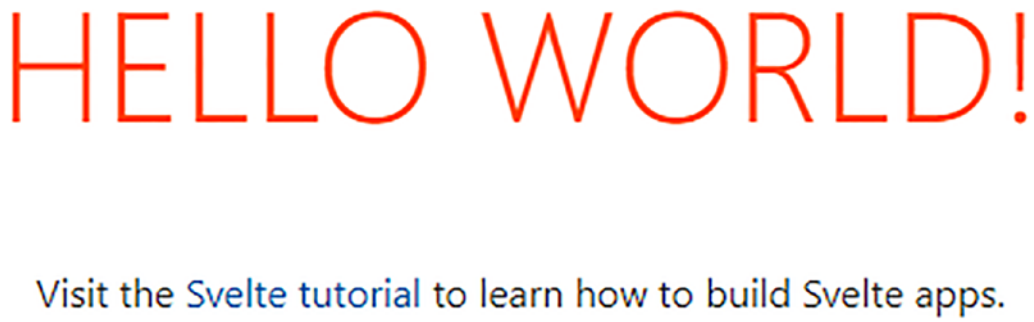


***Figure 1-4.*** *The base app for our project, running in a browser*

At this point, our site is now ready for us to develop into our store front end throughout this book. It was a simple exercise, but an important one – let's pause for a moment to review the steps we covered in this exercise in more detail.

## Reviewing the Changes Made

Setting up any site using Svelte is very easy – to do so, we first installed Node.js (and NPM) before extracting a copy of the custom template site from the code download. We then ran the typical `npm install && npm run dev` commands to install any required dependencies before launching the site in our browser.

With our starting site operational, now is an excellent time to take a quick look at the basic structure of a typical Svelte site and some of the basic principles of using Svelte in more detail.

# Exploring the Structure of Our Site

Remember how we worked through setting up a Svelte starter site at the start of this chapter, using the custom template provided by the Svelte developers? If we open the sveltebook folder we created earlier, Figure 1-5 shows us what to expect.